PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING — LEVEL 3 MINI-PROJECT

Software Architecture and Design Specification

for

Simple Shell — Command Line Interpreter

Item	Details
Project Title	Simple Shell - Command Line Interpreter
Group	Team 3
Course	UE24CS341A - Software Engineering
Authors	Rekha Dhorigol (PES1UG24CS370) / Rohan Rampura Praveen Kumar (PES1UG24CS382) / Samika Ojha (PES1UG24CS412) / Sanjana Subrahmanya Aithal (PES1UG24CS421)
Version	1.1
Status	Revised - Sprint 2 Implementation Baseline

Revision History

Version	Date	Author	Change Summary
1.0	2026-10-02	Team 3	Initial Software Architecture and Design Specification for Deliverables Part-1
1.1	2026-11-05	Team 3	Updated architecture descriptions to reflect the Sprint 2 implementation, including background process groups, background pipelines, process reaping, actual module interfaces and the current development toolchain. Corrected API return contracts, help behavior, data representations, and references

Approvals

Role	Name	Signature/Date
Course Coordinator	Dr. Sapna V M	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of Team 3's Simple Shell — Command Line Interpreter, implementing the requirements baselined in the Software Requirements Specification (SRS v1.3).

1.2 Scope

Covers the shell's internal architecture — command interpretation, foreground and background process execution, I/O redirection and pipelines, and command history/environment management — including component structure, sequence flows, internal module APIs, and security architecture. Full POSIX job-control commands (jobs, fg and bg) and terminal foreground process-group handoff are outside the current implementation scope. Does not cover OS kernel internals or terminal emulator implementation.

1.3 Audience

Developers (Team 3), QA/testers, and faculty evaluators.

1.4 Definitions

Shell, built-in, external command, foreground/background execution, background process management, redirection, pipeline, zombie process, environment variable, POSIX, IPC (inter-process communication), fd (file descriptor), SRS, RTM, STP, STRIDE. Full definitions are in Appendix 5.1 and SRS Appendix A.

2. Document Overview

2.1 How to Use This Document

Provides architectural deliverables including a UML component diagram, two UML sequence diagrams, internal module API definitions, and a lightweight threat model (STRIDE) for the shell's security architecture.

2.2 Related Documents

- SRS — Simple Shell Software Requirements Specification, v1.3
- STP — Simple Shell Software Test Plan, v1.1 (companion document)

3. Architecture

3.1 Goals & Constraints

Goals: a modular architecture where each of the four functional areas (F1–F4) is independently unit-testable (NFR-05); correct POSIX process/signal handling; robustness against malformed or adversarial input; no shell-injection vectors (SEC-01).

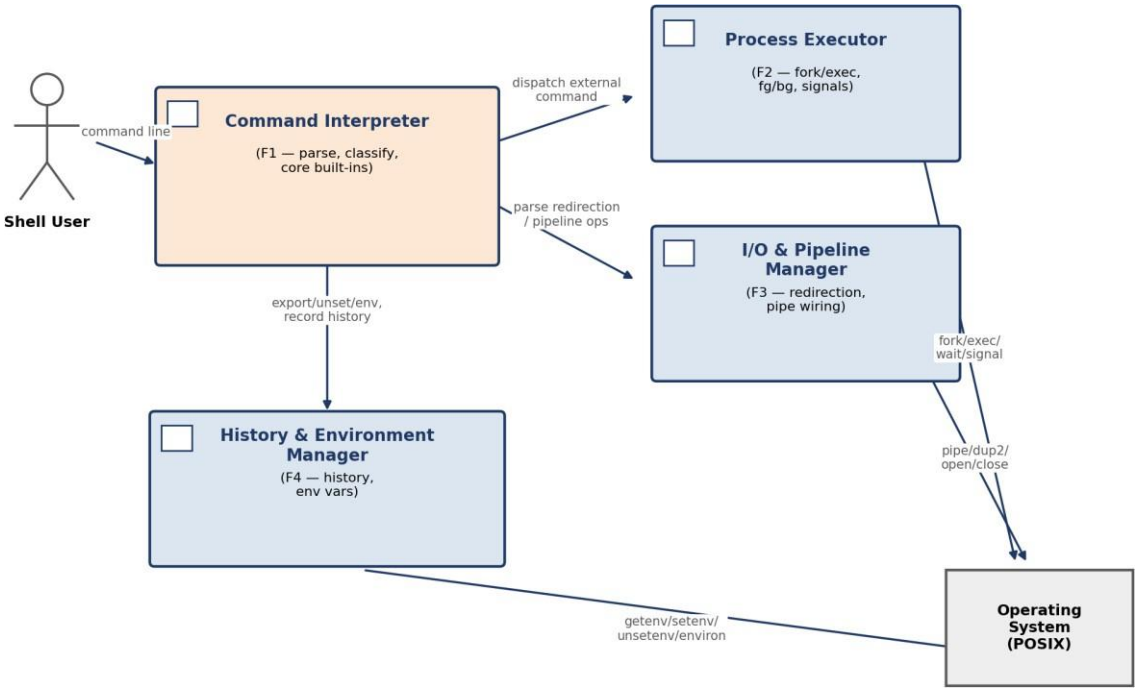
Constraints: C11/POSIX only; no external shell-building library may replace core parsing logic; build via GCC/Make; Agile delivery across six sprints with one functional area per owner. Memory safety will be validated using Valgrind and/or AddressSanitizer during the testing phase; these tools are not part of the current GitHub Actions workflow, and results will be recorded only after the checks are run.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Students / Learners	Architecture should make OS concepts (processes, fds, signals) visible and understandable, not hidden behind abstraction.
Command-line Users	Predictable, bash-like behaviour for the supported command subset.
Developers (Team 3)	Modularity and independent testability of F1–F4; clear interfaces between modules.
Faculty / Evaluators	Traceability from requirement to component to test case; evidence of individual contribution per module.

3.3 Component (UML) Diagram

(next page)



Solid arrows: control/data flow between the four functional modules and the OS.
All modules run in a single shell process; there is no network or database component.

3.4 Component Descriptions

- **Command Interpreter (F1, owner: Rekha):** Owns the prompt loop, input validation, pipeline splitting, command tokenizer, built-in/external command classification and built-in dispatcher. Implements the core built-ins cd, pwd, echo, exit and help, and dispatches history, export, unset and env through the History & Environment Manager services.
- **Process Executor (F2, owner: Rohan):** Executes standalone external commands using fork() and execvp(), waits for foreground children using waitpid(), and supports asynchronous background execution. Background standalone commands are isolated into separate process groups. Terminated background children are periodically reaped using waitpid() with WNOHANG during the shell's idle polling loop. Child processes restore default SIGINT handling.
- **I/O & Pipeline Manager (F3, owner: Samika):** Receives parsed commands and applies input/output redirection (<, >, >>). Creates multi-stage pipelines using pipe(), fork() and dup2(), closes unused file descriptors, and coordinates foreground and background pipeline execution. Background pipeline stages are placed in a shared process group, with synchronization before execution begins.
- **History & Environment Manager (F4, owner: Sanjana):** Maintains the bounded, in-memory command history and provides history retrieval/display services. Provides environment-variable services through env_set(), env_unset(), env_list() and env_print(), using the process environment so changes are inherited by subsequently created child processes.

3.5 Chosen Architecture Pattern and Rationale

Pattern: a layered module structure (Command Interpreter as a thin control layer over three independent service modules) combined with the classic Unix pipe-and-filter pattern for pipeline execution itself, where each pipeline stage is a separate OS process connected by anonymous pipes.

Rationale: the layered split lets each of the four functional areas be developed, unit-tested and graded independently (satisfying the one-owner-per-feature requirement and NFR-05's per-module coverage target), while pipe-and-filter is the natural, POSIX-native fit for how a pipeline must actually execute.

Rejected alternatives: a single monolithic parser+executor file was rejected because it would not support independent ownership or isolated unit testing. An event-driven/asynchronous architecture was rejected as unnecessary complexity for a synchronous, single-user interactive CLI with no concurrent external clients.

3.6 Technology Stack & Data Stores

- Language/Build: C11, compiled with GCC via Make.
- OS interfaces: POSIX system calls and library interfaces — fork, exec-family, wait/waitpid, pipe, dup2, signal/sigaction, poll.
- Data stores: No external database or persistent data store. The shell uses ParsedCommand (include/shell.h), a private fixed-size HistoryEntry array (src/history.c), and the POSIX process environment (environ). Pipeline stages are passed as an array of ParsedCommand pointers; no separate Pipeline structure or EnvironmentVariable structure is declared.
- Quality tooling: Custom C unit and integration test harnesses are executed through make test. Valgrind/AddressSanitizer and cppcheck are planned validation tools; their results shall only be reported after the corresponding checks have actually been run.
- SDLC tooling: Git/GitHub (version control), GitHub Projects (Agile board), GitHub Actions (CI/CD), and GCC/Make (build automation).

3.7 Risks & Mitigations

Risk	Mitigation
Resource exhaustion from repeated background jobs	FR-PE-06 periodic background-child reaping using poll() and waitpid() with WNOHANG. A configurable maximum concurrent-job count remains a possible future enhancement.
Pipe/fd leaks causing hangs on multi-stage pipelines	FR-IO-07 close-unused-descriptor discipline, verified by the pipeline resource test in tests/test_pipeline.c.
Buffer overflow from malformed/oversized input	SEC-02 bounded input handling; SEC-06 memory-safety validation using Valgrind/ASan during the testing phase.

3.8 Traceability to Requirements

Requirement Group	Primary Component
FR-CI-01..07 (F1)	Command Interpreter
FR-PE-01..07 (F2)	Process Executor
FR-IO-01..07 (F3)	I/O & Pipeline Manager
FR-HE-01..07 (F4)	History & Environment Manager
SEC-01, SEC-02, SEC-06	Command Interpreter (input handling) — and all modules for memory safety
SEC-03	History & Environment Manager
SEC-04, SEC-05	Process Executor / I/O & Pipeline Manager
NFR-01, NFR-02, NFR-03	Process Executor / I/O & Pipeline Manager
NFR-04 (Usability)	Command Interpreter (prompt and help design)
NFR-05 (Maintainability)	All four modules (per-module unit coverage)

3.9 Security Architecture

Lightweight STRIDE-style threat model, scoped to a single-user local shell process (no network, no multi-user authentication, so Spoofing and Repudiation are largely not applicable to this scope):

Threat	Example in Simple Shell	Mitigation
Spoofing	Not applicable — single local invoking user, no identity to spoof.	—
Tampering	Crafted command line attempting a buffer overflow in the tokenizer.	SEC-02 bounded input copy; SEC-06 ASan/Valgrind checks.
Repudiation	Not applicable — no multi-user audit trail is in scope.	—
Information Disclosure	Environment-variable values leaking via unintended output or logs.	SEC-03: no implicit print/log of env values.
Denial of Service	Zombie-process accumulation or fd exhaustion from repeated pipelines.	FR-PE-06 reaping; FR-IO-07 descriptor closing.
Elevation of Privilege	Shell-injection via system()/popen() executing beyond the typed command.	SEC-01: argument-vector exec only, no shell interpolation.

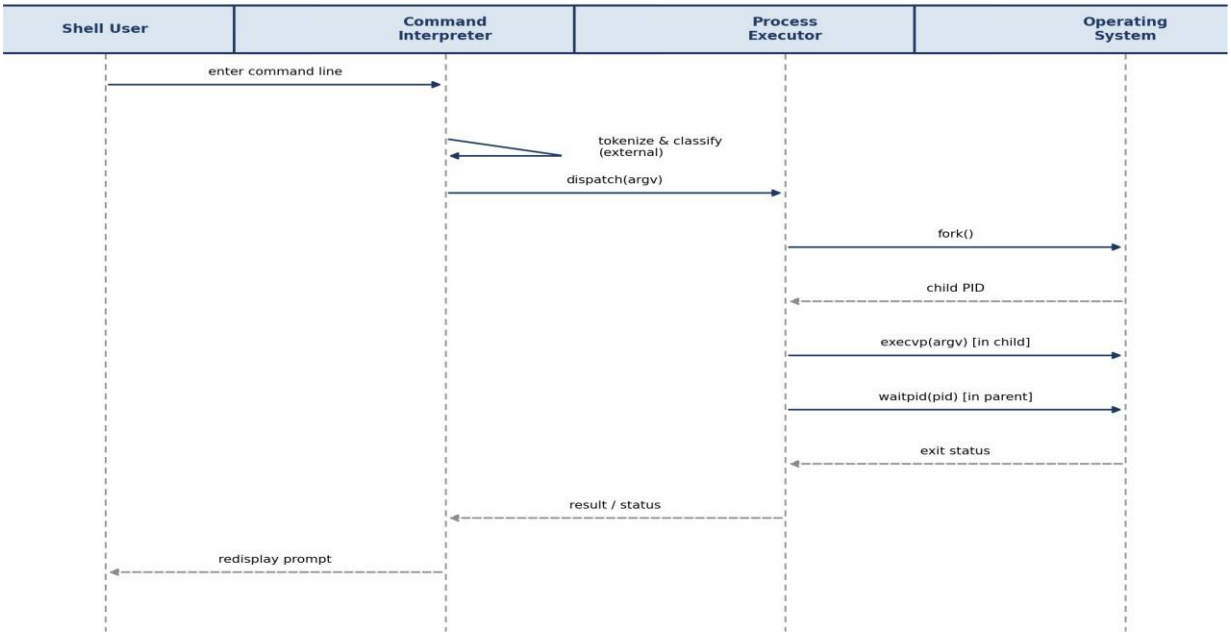
4. Design

4.1 Design Overview

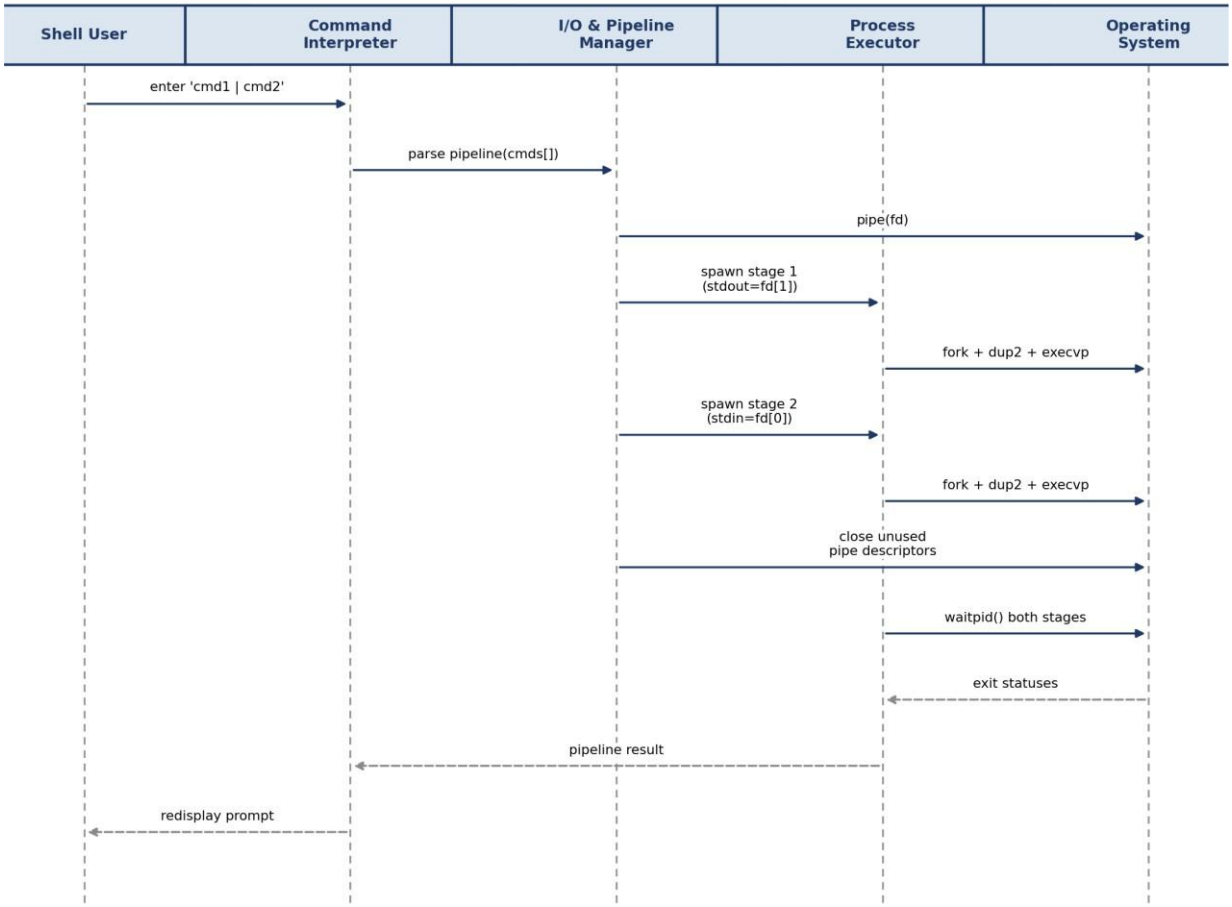
The shell is designed as four independently testable modules coordinated by the Command Interpreter through one command-processing cycle: read → tokenize → classify → dispatch → execute → display. Each module exposes a small C function-level API (4.3) so modules can be unit-tested with mocked calls to the others.

4.2 UML Sequence Diagrams

Sequence Diagram 1 — Execute External Command (Foreground, UC-02)



Sequence Diagram 2 — Execute Command Pipeline (UC-05)



4.3 API Design

Internal module interfaces (the shell has no network API; these are the C function-level contracts between components, covering more than the minimum two required):

Interface 1: Command Interpreter → Process Executor

- int executor_run(const ParsedCommand *cmd);
- Input: pointer to a parsed command containing the argument vector, background flag and optional redirection information.
- Output: command exit status. Returns the child's exit status for foreground execution and 0 when a background command is successfully started. Returns 1 for fork/wait/process-management errors and 2 for an empty/invalid command. If execvp() fails, the child exits with status 127 when the command is not found (ENOENT), or 126 for other execution failures.
- Errors: diagnostic messages are written to standard error. Fork and wait failures are handled by the executor; the parent does not receive the child's execvp() errno directly.

Interface 2: Command Interpreter → I/O & Pipeline Manager

- int pipeline_execute(ParsedCommand *stages[], int stage_count);
- Input: ordered array of parsed commands forming the pipeline (stage_count ≤ 5 per FR-IO-05).
- Output: exit status of the last stage.
- Returns the exit status of the final pipeline stage for foreground execution, 0 when a background pipeline is successfully started, 1 for pipe/fork/process-management failures, and 2 for invalid pipeline input.
- Errors: pipe(), fork() and process-group setup failures return 1; invalid pipeline arguments return 2. Partially started children are terminated and waited for during cleanup. An execvp() failure in a stage is reported by that child, which exits with status 127.

Interface 3: Command Interpreter → History & Environment Manager

- void history_add(const char *command_line);
- void history_print(void);
- int history_count(void);
- const char *history_get(int position);
- unsigned long history_get_number(int position);
- void history_clear(void);
- int env_set(const char *name, const char *value);
- int env_unset(const char *name);
- char **env_list(void);
- void env_print(void);
- env_print()

4.4 Error Handling, Logging & Monitoring

User-facing diagnostics are written to standard error and generally use the shell: prefix, followed by a command-specific or operation-specific description. System-call failures may use perror() to include the corresponding operating-system error description. No sensitive information — environment-variable values or file contents — is logged unless explicitly requested via env (SEC-03). No persistent log file is required for this scope; diagnostic output goes to stderr only. Monitoring is manual: the test harness records pass/fail per test case (see the companion Test Plan); no runtime telemetry service is needed given the shell has no network interface (SRS 3.3).

4.5 UX Design

- Prompt format: simple-shell>, redisplayed after command processing, including after foreground completion and background launch.
- Consistent error-message convention across all modules (4.4).
- history built-in prints a numbered list of current-session commands.
- help built-in lists the supported built-ins and explains that external commands are executed through the process executor; it does not provide individual usage notes.
- No GUI; output relies on plain, readable text rather than color, for terminal portability.

4.6 Open Issues & Next Steps

- Persistent (cross-session) history — explicitly out of scope per SRS 2.4; candidate post-submission enhancement.
- Full interactive job-control commands (jobs, fg, bg) and terminal foreground-process-group handoff are outside the current implementation scope; they may be considered as future enhancements.
- Command-line editing/autocompletion — out of scope for this course project.
- Alias support (alias/unalias) — out of scope; future enhancement.

5. Appendices

5.1 Glossary

See SRS Appendix A for the full glossary (built-in command, external command, foreground/background execution, redirection, pipeline, Background process management, zombie process, environment variable). Additional terms used in this document: IPC (inter-process communication); fd (file descriptor); STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — a threat-modeling framework).

5.2 References

- Simple Shell SRS, v1.3 (companion document)
- Simple Shell Software Test Plan, v1.1 (companion document)
- POSIX.1-2017 (IEEE Std 1003.1), process/signal/I/O sections

5.3 Tools

Draw.io/PlantUML-equivalent diagrams (rendered here as generated figures), GCC/Make, Valgrind/AddressSanitizer, cppcheck, GitHub Actions.

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING — LEVEL 3 MINI-PROJECT

Software Requirements Specification

for

Simple Shell — Command Line Interpreter

Item	Details
Project Title	Simple Shell - Command Line Interpreter
Group	Team 3
Course	UE24CS341A - Software Engineering
Technology	C11 / POSIX / Linux
Methodology	Agile
Team	Rekha Dhorigol (PES1UG24CS370) / Rohan Rampura Praveen Kumar (PES1UG24CS382) / Samika Ojha (PES1UG24CS412) / Sanjana Subrahmanya Aithal (PES1UG24CS421)
Version	1.3 - Implementation-aligned requirements baseline

Revision History

Version	Date	Description
1.0	2026-09-07	Initial requirements baseline
1.1	—	Revised requirements baseline: fixed limits (1024-character input, 64 argv entries including the command name, 50 history entries, 5 pipeline stages), simplified core built-in set, refined scope
1.2	2026-10-02	Added NFR-04/NFR-05 (§6.2) and Security Objectives (§6.4.1) to meet Deliverables Part-1 minimums (≥5 NFRs, ≥2 security objectives, ≥2 security requirements); prepared for combined submission with the SAD and Test Plan
1.3	2026-11-05	Updated terminology and implementation-aligned descriptions following Sprint 2 integration; clarified background process management, built-in command scope. Clarified argv limit semantics, corrected in-memory structure descriptions, and updated implementation/test traceability

Approvals

Role	Name	Signature/Date
Course Coordinator	Dr. Sapna V M	

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for Team 3's Simple Shell project. It provides the requirements baseline against which the implementation, testing and validation deliverables are assessed.

1.2 Intended Audience

The document is intended for the project team, testers, faculty and evaluators. It defines what the system shall do and provides the basis for later design, implementation, testing and validation.

1.3 Product Scope

The Simple Shell is a lightweight command-line interpreter. It reads a command, parses it, executes supported built-ins or external programs, and provides foreground/background execution, redirection, pipelines, current-session history and environment-variable operations.

1.4 References

- SE Level 3 Mini-Project — Student Submission Guide & Complete Exemplar (Coffee Kiosk), PES University
- Jackfruit — Software Engineering Project Guidelines 2026, PES University
- POSIX.1-2017, relevant process, signal and I/O interfaces
- SE Mini-Project Deliverables Part-1 brief (ETA 28 Sept 2026) and SRS/SAD/Test Plan templates, PES University

2. Overall Description

2.1 Product Perspective

The shell is a standalone terminal application. It consists of four functional areas: F1 Command Interpretation & Built-ins, F2 Process Execution & Background Handling, F3 I/O Redirection & Pipelines, and F4 Command History & Shell Environment. The modules use standard C/POSIX facilities and do not require a database, server or network service.

2.2 Product Functions

- Display a prompt and read one command line.
- Tokenize and parse the command and arguments.
- Execute the core built-ins cd, pwd, echo, exit and help, and provide the additional current-session built-ins history, export, unset and env.
- Execute external programs using POSIX processes.
- Support foreground and background execution using '&'.
- Support input (<), output (>) and append (>>) redirection.
- Support pipelines containing up to five commands.
- Store and display commands entered during the current session.
- Support export, unset and env for current-session environment variables.
- Report invalid input and execution errors without crashing.

2.3 User Classes

User Class	Description
Students / Learners	Use the shell to understand basic OS and process concepts.
Command-line Users	Use the shell interactively with standard user privileges.
Developers / Testers	Build, inspect and test the source code and modules.
Faculty / Evaluators	Assess functional correctness, SDLC evidence and individual contribution.

2.4 Scope

In Scope	Out of Scope
Interactive prompt and input	GUI
Parsing and command execution	Remote/network execution/ Full interactive POSIX job control (jobs, fg, bg) and terminal foreground-process-group handoff.
cd, pwd, echo, exit, help	Full shell scripting language
Foreground/background execution	Root/admin management
<, > and >>	Persistent cross-session history
Pipelines up to 5 commands	Loops, conditionals and shell functions
Current-session history	Command recall/editing
export, unset and env	Database/network services

2.5 Operating Environment

- Linux/POSIX-compatible operating system; Ubuntu/Linux used for development and testing.
- C11 compiled with GCC.
- Interactive terminal (TTY); no GUI.

2.6 Constraints

- Use POSIX APIs such as fork(), exec-family calls, wait/waitpid(), pipe(), dup2() and signal handling.
- Core shell parsing shall be implemented by the team.

- Use Git/GitHub and the agreed Agile workflow.
- Dynamic memory and file descriptors shall be released appropriately.
- The four functional areas shall remain modular and testable.

2.7 Assumptions and Dependencies

- Single-user Linux environment with standard POSIX libraries.
- External commands exist on PATH or are supplied using a valid executable path.
- No third-party runtime service is required.

3. External Interface Requirements

3.1 User Interface

- A text prompt such as 'simple-shell> ' shall be displayed.
- The user enters commands through standard keyboard input.
- Output and errors use standard output/error streams.
- Invalid input produces an error and returns to the prompt.
- The history built-in lists current-session commands with numeric indices.

3.2 Software Interfaces

Interface	Purpose
fork(), execvp()/execve(), waitpid()	Create, execute and wait for processes.
open(), close(), dup2(), pipe()	Implement redirection and pipelines.
signal()/sigaction()	Handle interrupts and child-process signals.
C standard library	String, memory and argument handling.
getenv(), setenv(), unsetenv(), environ	Environment-variable operations.

3.3 Communications Interfaces

Not applicable. The shell is a local application and has no network communication requirement.

4. Analysis Models

4.1 Actors

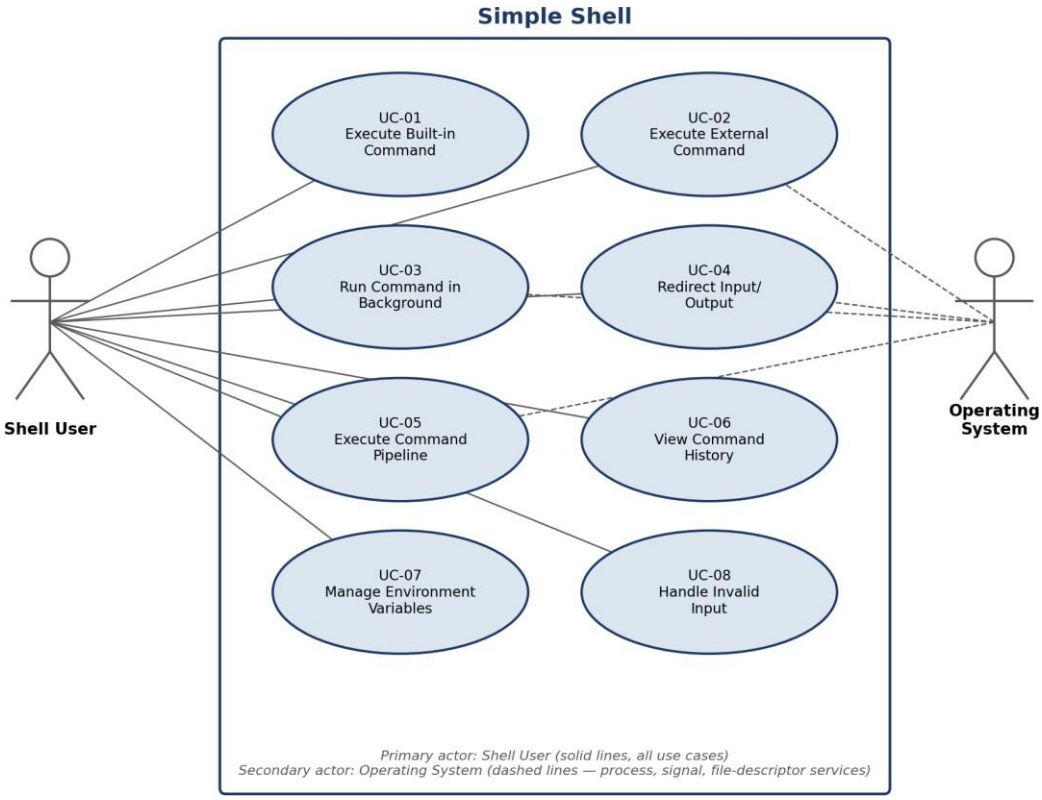
The primary actor is the Shell User. The Operating System provides process, file and signal services.

4.2 Use Case Summary

ID	Use Case	Outcome
UC-01	Execute Built-in Command	Supported built-in executes; prompt returns.
UC-02	Execute External Command	External program executes; shell resumes.
UC-03	Run Command in Background	Shell returns to prompt without waiting.
UC-04	Redirect Input/Output	stdin/stdout is connected to a specified file.
UC-05	Execute Command Pipeline	Two to five commands are connected by pipes.
UC-06	View Command History	Current-session commands are displayed with indices.
UC-07	Manage Environment Variables	Variable is set, removed or listed.
UC-08	Handle Invalid Input	Error is shown and shell continues.

4.3 Use Case Diagram

(next page)



The requirements-level diagram contains the Shell User and Operating System actors and the eight use cases above. Detailed sequence diagrams are produced as part of the SAD (see the companion Software Architecture and Design Specification, Section 4.2).

5. System Features

Ownership rule: each feature has one primary owner for end-to-end functionality, but all four team members contribute to requirements, design, implementation, testing, review and validation.

Feature	Primary Owner	Scope
F1 Command Interpretation & Core Built-in Commands	Rekha Dhorigol	Parsing, classification, core built-ins.
F2 Process Execution & Background Handling	Rohan Rampura Praveen Kumar	External commands, foreground/background execution and basic signal handling.
F3 I/O Redirection & Pipelines	Samika Ojha	<, >, >> and pipelines.
F4 Command History & Shell Environment	Sanjana Subrahmanya Aithal	Current-session history and environment operations.

5.1 Command Interpretation & Core Built-ins (F1)

Owner: Rekha Dhorigol

ID	Requirement	Priority	Verification
FR-CI-01	The shell shall display a prompt and accept one input line of at most 1024 characters.	High	Functional test
FR-CI-02	The shell shall tokenize the input into a command and zero or more arguments, with at most 64 argv entries in total, including the command name.	High	Unit test
FR-CI-03	The shell shall classify a parsed command as a supported built-in or external command.	High	Unit test
FR-CI-04	The shell shall implement cd, pwd, echo, exit and help.	High	Functional test
FR-CI-05	The shell shall display an error for empty, malformed or unrecognised input and return to the prompt.	High	Negative test
FR-CI-06	The shell shall reject input longer than 1024 characters without crashing or corrupting shell state.	High	Boundary test
FR-CI-07	The exit built-in shall terminate the shell cleanly and release allocated resources.	Medium	Functional + memory test

5.2 Process Execution & Background Handling (F2)

Owner: Rohan Rampura Praveen Kumar

ID	Requirement	Priority	Verification
FR-PE-01	The shell shall create a child process using fork() for each external command.	High	Unit test
FR-PE-02	The child shall execute the requested program using an exec-family call and the parsed arguments.	High	Functional test
FR-PE-03	For foreground execution, the shell shall wait for the child before displaying the next prompt.	High	Functional test
FR-PE-04	For a command ending with &, the shell shall start the command and return to the prompt without waiting.	High	Functional test
FR-PE-05	The shell shall report when a foreground command terminates abnormally.	Medium	Functional test
FR-PE-06	The shell shall reap terminated background children to prevent zombie processes.	High	Integration test
FR-PE-07	The shell shall handle Ctrl+C so that the shell itself is not terminated by an interrupt intended for a foreground command.	High	Signal test

5.3 I/O Redirection & Pipelines (F3)

Owner: Samika Ojha

ID	Requirement	Priority	Verification
FR-IO-01	The shell shall redirect stdout using > and replace existing file contents.	High	Functional test
FR-IO-02	The shell shall redirect stdout using >> and append to an existing file.	High	Functional test
FR-IO-03	The shell shall redirect stdin using < from a specified file.	High	Functional test
FR-IO-04	The shell shall support a two-command pipeline using |.	High	Integration test
FR-IO-05	The shell shall support pipelines containing up to five commands.	Medium	Integration test
FR-IO-06	The shell shall display an error and skip execution when a required redirection file cannot be opened.	High	Negative test
FR-IO-07	The shell shall close unused pipe descriptors to avoid descriptor leaks and hangs.	High	Resource test

5.4 Command History & Shell Environment (F4)

Owner: Sanjana Subrahmanya Aithal

ID	Requirement	Priority	Verification
FR-HE-01	The shell shall store successfully parsed command lines in an in-memory history buffer of at most 50 entries.	Medium	Unit test
FR-HE-02	The history built-in shall display stored commands in order with numeric indices.	Medium	Functional test
FR-HE-03	When 50 entries are stored, adding a new entry shall remove the oldest entry.	Medium	Unit test
FR-HE-04	The export built-in shall set or update an environment variable for the current session.	High	Functional test
FR-HE-05	The unset built-in shall remove an environment variable from the current session.	Medium	Functional test
FR-HE-06	The env built-in shall list environment variables currently available to the shell.	Medium	Functional test
FR-HE-07	Environment-variable changes shall be visible to subsequently created child processes.	High	Integration test

6. Non-Functional Requirements

6.1 Performance

ID	Requirement	Verification
NFR-01	The shell shall redisplay the prompt within 1 second after a normal foreground command completes, under the reference test environment.	Performance test
NFR-02	The shell shall be ready to accept input within 1 second of launch under the reference test environment.	Performance test
NFR-03	A pipeline of up to five lightweight test commands shall execute without deadlock or indefinite blocking.	Integration test

6.2 Usability & Maintainability

ID	Requirement	Category	Verification
NFR-04	A first-time user shall be able to successfully run at least 3 of the 5 core built-ins (cd, pwd, echo, exit, help) within 2 minutes of first seeing the prompt, without external documentation.	Usability	Usability walkthrough
NFR-05	Each of the four functional modules (F1–F4) shall be unit-testable in isolation and shall achieve at least 70% line coverage in its own unit test suite.	Maintainability	Code review + coverage report

6.3 Safety

- The shell shall not provide hidden destructive operations.
- Terminated child processes shall be reaped appropriately.

6.4 Security

6.4.1 Security Objectives

- SEC-OBJ-01: Prevent command or argument injection from granting execution beyond what the invoking user explicitly typed — no shell-interpolation functions and no implicit privilege escalation.
- SEC-OBJ-02: Ensure all parsing, tokenization and history/environment buffer operations are memory-safe, so malformed, oversized or adversarial input cannot corrupt shell state, crash the process, or be leveraged for memory-corruption exploits.
- SEC-OBJ-03: Protect the confidentiality of environment-variable values and command history from unintended disclosure through shell output, error messages or logs, exposing them only when the user explicitly requests them.

6.4.2 Security Requirements

ID	Requirement	Validation
SEC-01	The shell shall not use system(), popen() or equivalent shell-interpolation functions to execute user commands; commands shall be passed as argument vectors to exec.	Code review + negative test
SEC-02	The shell shall reject input longer than 1024 characters and shall not copy it beyond allocated bounds.	Boundary test + static analysis
SEC-03	The shell shall not print or log environment-variable values unless the user explicitly requests them through env.	Code review
SEC-04	The shell shall handle child-process signals without terminating unexpectedly and shall safely reap terminated children.	Signal/negative test
SEC-05	Redirection shall occur only when the user explicitly supplies <, > or >> and shall use the invoking user's OS permissions.	Functional + negative test
SEC-06	Parsing and history memory shall be released exactly once and checked for leaks or invalid access.	Valgrind/ASan + code review

6.5 Quality Attributes

- Correctness: defined commands shall produce expected results in the test cases.
- Reliability: invalid input shall not crash the shell.
- Maintainability: the four functional areas shall remain modular.
- Testability: each functional area shall have focused unit/integration tests.

6.6 Domain Rules

- The shell operates under the invoking user's OS permissions.
- History is current-session only and limited to 50 entries.
- A pipeline may contain at most five commands.
- Only features explicitly listed in this SRS are required.

7. Other Requirements

7.1 Memory and Resource Management

- Dynamic memory used for parsing and history shall be released appropriately.
- File descriptors created for redirection and pipelines shall be closed when no longer required.
- Child processes shall be waited for or reaped as appropriate.

7.2 Fixed Limits

Item	Limit
Maximum command-line length	1024 characters
Maximum argv entries per command (including command name)	64
Maximum history entries per session	50
Maximum pipeline stages	5 commands

7.3 Portability

The project targets Linux/POSIX systems. Portability to non-POSIX operating systems is not required.

7.4 External Dependencies

No database, cloud service or network service is required at runtime. The shell depends on the Linux/POSIX environment and standard C/POSIX libraries.

8. Initial Validation Specification

Validation will be performed as features are implemented. The verification method for every functional, security and non-functional requirement is stated in Sections 5 and 6.

Validation Areas

Area	Checks
Parsing / built-ins	Prompt, tokenization, built-ins, invalid input and 1024-character boundary.
Processes	Foreground/background execution, abnormal termination, child reaping and Ctrl+C.
Redirection	<, > and >> with valid and inaccessible files.
Pipelines	Two-stage and up to five-stage pipelines; no deadlock; descriptors closed.
History	Recording, ordered display, 50-entry boundary and oldest-entry removal.
Environment	export, unset, env and visibility in child processes.
Security / robustness	No shell interpolation, bounded input, signal handling and memory checks.
Performance	Startup and prompt-response measurements under the same reference environment.

Reference Test Environment

Ubuntu/Linux system, GCC and a standard terminal session. Measurements shall be recorded on the same reference machine when possible.

9. Project Plan

9.1 Work Breakdown Structure

WP	Work Package	Coordination
WP1	Requirements baseline	All members
WP2	Architecture and design	All members
WP3	F1 Command Interpretation	Rekha
WP4	F2 Process Execution	Rohan
WP5	F3 Redirection & Pipelines	Samika
WP6	F4 History & Environment	Sanjana
WP7	Integration, CI and security checks	All members
WP8	Final validation, report and demo	All members

9.2 Six-Sprint Plan

Work Package	S1	S2	S3	S4	S5	S6
WP1 Requirements	X					
WP2 Architecture/Design	X	X				
WP3 F1		X				
WP4 F2			X			
WP5 F3				X		
WP6 F4				X		
WP7 Integration/CI/Security					X	
WP8 Validation/Report/Demo						X

(Updated to 2 sprints)

Work Package	S1	S2
WP1 Requirements	X	
WP2 Architecture/Design	X	
WP3 F1	X	
WP4 F2		X
WP5 F3		X
WP6 F4		X
WP7 Integration/CI/Security		X
WP8 Validation/Report/Demo		

10. Tool Stack

Activity	Tool	Evidence
Version control	Git + GitHub	Repository and commit history
Agile planning	GitHub Projects	Backlog and sprint board
Build	GCC + Make	Makefile and build output
CI/CD	GCC / Make / custom C test harness	Test source and make test output
Testing	Custom C test harnesses (GCC / Make)	Test source and make test output
Static analysis	cppcheck or suitable C tool	Analysis report
Memory/robustness	Valgrind or AddressSanitizer	Memory/leak report
Design	Draw.io / StarUML / PlantUML	UML diagrams

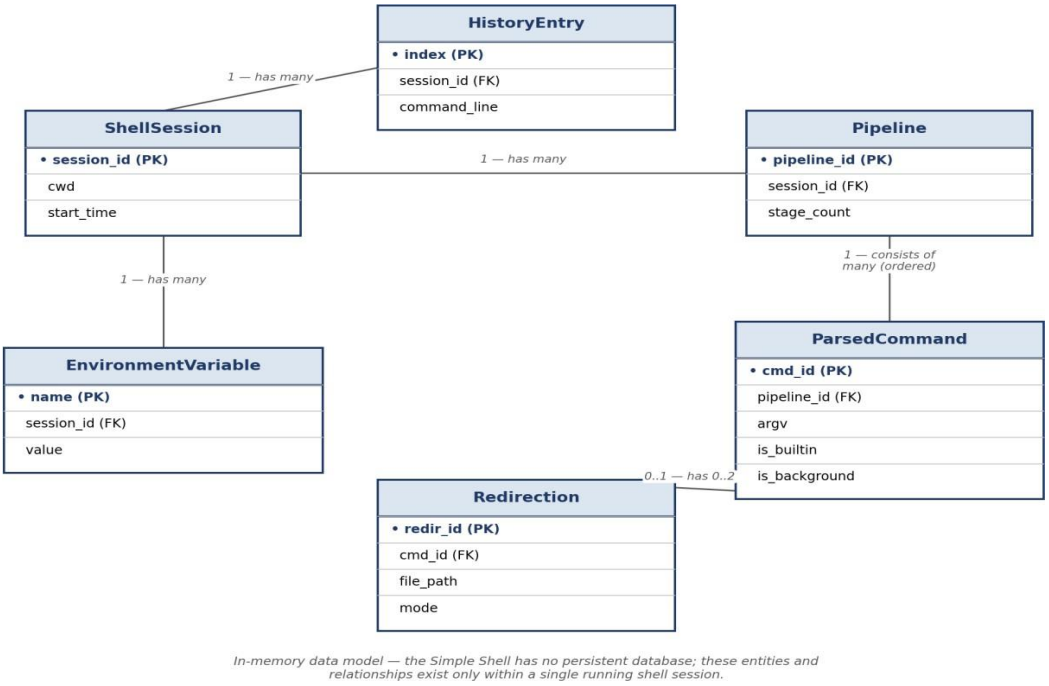
Appendix A: Glossary

Term	Definition
Built-in command	A command implemented directly within the shell process, such as cd or exit.
External command	A program executed by creating a child process and using an exec-family call.
Foreground execution	Execution in which the shell waits for the command to finish.
Background execution	Execution in which the shell starts the command and returns to the prompt without waiting.
Redirection	Connecting standard input/output to a file using <, > or >>.
Pipeline	Two to five commands connected so stdout of one becomes stdin of the next.
Job control	Background process management: Starting commands asynchronously, isolating background processes into process groups, and reaping terminated child processes. Full interactive job-control commands such as jobs, fg and bg are outside the current project scope.
Zombie process	A terminated child whose exit status has not yet been collected by its parent.
Environment variable	A named value in the process environment that may be inherited by child processes.

Appendix B: Field Layouts

The shell has no external database. The following layouts describe the actual in-memory representations used by the current implementation.

(Next page)



B.1 History Entry

Field	Type	Description	Mandatory
number	unsigned long number	Monotonically increasing command number displayed by the history built-in.	Y
command	char[1025]	Stored command line, limited to 1024 characters plus terminating null character.	Y

B.2 Parsed Command

Field	Type	Description	Mandatory
argv	char*[]	NULL-terminated argument vector; at most 64 entries in total, including argv[0] (the command name).	Y
is_builtin	bool	Whether the command is a supported built-in.	Y
input_file	char*	Input redirection path, if present.	N
output_file	char*	Output redirection path, if present.	N
append_mode	bool	True when >> is used.	N
is_background	bool	True when the command uses background execution.	Y

B.3 Pipeline

Parameter	Type	Description	Required
stages	ParsedCommand *stages[]	Caller-provided array of pointers to parsed pipeline-stage commands, passed to pipeline_execute(); no standalone Pipeline structure is declared.	Y
stage_count	int	Number of stages; valid range is 1–5.	Y

Pipeline Representation: Pipeline stages are represented as an array of pointers to ParsedCommand objects. The pipeline_execute() interface accepts this array and the number of stages. No separate Pipeline structure is declared.

B.4 Process Environment Representation

Representation	Type	Description	Required
environ	char **	POSIX process environment array; each entry is a NAME=VALUE string. env_list() returns environ; env_set() and env_unset() use setenv() and unsetenv(). No custom EnvironmentVariable structure is declared.	Y
value	char*	Environment variable value.	Y

Appendix C: Requirement Traceability Matrix

The RTM is updated to identify the current architecture/design references, implementation source files and planned test-case IDs. Test execution outcomes and measured coverage remain pending and are recorded in the companion STP/test evidence after validation.

Req ID	Brief Description	Arch.	Design	Code	Test
FR-CI-01	Prompt; input ≤1024 chars	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-01
FR-CI-02	Tokenize; max 64 argv entries incl. command	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-02
FR-CI-03	Classify built-in/external	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-01
FR-CI-04	Core built-ins	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-01
FR-CI-05	Error handling	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-02
FR-CI-06	Reject >1024 chars	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-CI-02
FR-CI-07	Clean exit	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h; include/parser.h	TC-SEC-02
FR-PE-01	fork()	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-01
FR-PE-02	exec-family	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-01
FR-PE-03	Foreground wait	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-01
FR-PE-04	Background execution	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-02
FR-PE-05	Abnormal termination	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-01
FR-PE-06	Reap children	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-02
FR-PE-07	Ctrl+C handling	SAD 3.4	SAD 4.3	src/executor.c; src/main.c; include/executor.h	TC-PE-01
FR-IO-01	Output redirection >	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-01
FR-IO-02	Append redirection >>	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-01
FR-IO-03	Input redirection <	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-01
FR-IO-04	Two-command pipeline	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-02
FR-IO-05	Pipeline max 5 stages	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-02
FR-IO-06	Redirection error	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-01
FR-IO-07	Close unused descriptors	SAD 3.4	SAD 4.3	src/main.c; src/pipeline.c; include/pipeline.h; include/shell.h	TC-IO-02
FR-HE-01	History max 50 entries	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-01
FR-HE-02	Display history	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-01
FR-HE-03	Remove oldest when full	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-01
FR-HE-04	export	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-02
FR-HE-05	unset	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-02
FR-HE-06	env	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-02
FR-HE-07	Environment reaches children	SAD 3.4	SAD 4.3	src/main.c; src/history.c; src/env_manager.c; include/history.h; include/env_manager.h	TC-HE-02
SEC-01	No shell interpolation	SAD 3.4	SAD 4.3	src/executor.c; src/pipeline.c (execvp argument vectors)	TC-SEC-01
SEC-02	Bounded input	SAD 3.4	SAD 4.3	src/main.c; src/parser.c; include/shell.h	TC-CI-02; TC-SEC-02

SEC-03	No unintended env exposure	SAD 3.4	SAD 4.3	src/main.c; src/env_manager.c	TC-HE-02
SEC-04	Safe child signal handling	SAD 3.4	SAD 4.3	src/main.c; src/executor.c; src/pipeline.c	TC-PE-01; TC-PE-02
SEC-05	Explicit redirection	SAD 3.4	SAD 4.3	src/executor.c; src/pipeline.c; src/main.c	TC-SEC-01; TC-IO-01
SEC-06	Safe memory handling	SAD 3.4	SAD 4.3	src/parser.c; src/history.c; src/main.c	TC-SEC-02
NFR-01	Prompt response ≤1s	SAD 3.4	SAD 4.3	src/main.c; benchmark.sh	TC-NFR-01
NFR-02	Startup ≤1s	SAD 3.4	SAD 4.3	src/main.c; benchmark.sh	TC-NFR-01
NFR-03	Pipeline up to 5 stages; no deadlock	SAD 3.4	SAD 4.3	src/pipeline.c; tests/test_pipeline.c	TC-NFR-02
NFR-04	Usability — built-in discoverability	SAD 3.4	SAD 4.3	src/main.c (prompt/help/built-in dispatch)	TC-NFR-02
NFR-05	Maintainability — 70% module coverage	SAD 3.4	SAD 4.3	src/parser.c; src/executor.c; src/pipeline.c; src/history.c; src/env_manager.c; Makefile	Per-module coverage report (pending)

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING — LEVEL 3 MINI-PROJECT

Software Test Plan (STP)

for

Simple Shell — Command Line Interpreter

Item	Details
Project Title	Simple Shell - Command Line Interpreter
Group	Team 3
Course	UE24CS341A - Software Engineering
Authors	Rekha Dhorigol (PES1UG24CS370) / Rohan Rampura Praveen Kumar (PES1UG24CS382) / Samika Ojha (PES1UG24CS412) / Sanjana Subrahmanya Aithal (PES1UG24CS421)
Version	1.1
Status	Revised - Sprint 2 Implementation Baseline
Date	2026-11-05

Version	Date	Description
1.0	2026-10-02	Initial test plan
1.1	2026-11-05	Updated test strategy, test tooling, implementation references and test scope to match the Sprint 2 integrated baseline. Aligned scope, references, test tooling, CI limitations, security checks and traceability

1. Introduction

Purpose: This document defines the test plan for the Simple Shell project. It outlines objectives, scope, strategy, resources, schedule and responsibilities for testing the four functional areas (F1–F4) plus the security and performance requirements in the SRS.

Scope: Testing covers command interpretation and built-ins, foreground/background process execution and basic background-process management, I/O redirection and pipelines, and command history/environment management. Full interactive POSIX job control (jobs, fg, bg) and terminal foreground-process-group handoff are outside implementation scope. OS kernel internals, terminal-emulator behavior, and the correctness of external programs invoked by the shell (e.g. ls, cat) are excluded.

References: Simple Shell SRS v1.3, Simple Shell SAD v1.1, POSIX.1-2017.

Definitions: SRS (Software Requirements Specification), SAD (Software Architecture and Design Specification), RTM (Requirements Traceability Matrix), fd (file descriptor), NFR (Non-Functional Requirement), SEC (Security Requirement).

2. Test Items

- Command Interpreter module (F1) — prompt loop, tokenizer, classifier, dispatcher, core built-ins
- Process Executor module (F2) — fork/exec/wait, foreground/background execution, signal handling
- I/O & Pipeline Manager module (F3) — redirection, multi-stage pipelines
- History & Environment Manager module (F4) — command history, export/unset/env

3. Features to be Tested

Features mapped to SRS requirement IDs (SRS v1.3, Sections 5–6):

- FR-CI-01..07 — Command interpretation and core built-ins (cd, pwd, echo, exit, help)

- FR-PE-01..07 — External process execution, foreground/background, signal handling
- FR-IO-01..07 — Input/output/append redirection and pipelines up to 5 stages
- FR-HE-01..07 — Command history (50-entry bound) and export/unset/env
- SEC-01..06 — No shell interpolation, bounded input, no unintended env exposure, safe signal handling, explicit redirection only, safe memory handling
- NFR-01..05 — Prompt-response time, startup time, pipeline throughput, usability of built-ins, per-module maintainability/coverage

4. Features Not to be Tested

- OS kernel process-scheduling internals (assumed correct).
- Terminal emulator / TTY driver behaviour.
- Correctness of external programs themselves (e.g. ls, cat, grep) — only the shell's invocation, argument passing and redirection/piping around them is tested.
- Persistent cross-session history — explicitly out of scope per SRS 2.4.
- Graphical user interface — none exists.

5. Test Approach / Strategy

Levels

- Unit tests — per module, using the project's custom C test harnesses and assertions against module-level functionality and interfaces (SAD 4.3). The harnesses are built and run through Make; the project does not currently use Check or CMocka.
- Integration tests — cross-module flows, e.g. a pipeline spanning the Command Interpreter, I/O & Pipeline Manager and Process Executor.
- System tests — end-to-end interactive shell sessions covering UC-01..UC-08.
- Acceptance tests — faculty demonstration walkthrough of the eight use cases.

Types

- Functional testing — core FR-CI/PE/IO/HE behaviour.
- Negative / boundary testing — malformed input, 1024-character input and 64 argv-entry limits (including the command name), inaccessible redirection targets.
- Regression testing — the existing make test suite is run locally and by the GitHub Actions workflow on pushes and pull requests to main.
- Performance testing — NFR-01..03 response-time and throughput measurements.
- Usability testing — NFR-04 built-in discoverability walkthrough.
- Memory/robustness testing — AddressSanitizer and/or Valgrind runs are planned per SEC-06 and NFR-05; these checks are not currently part of the GitHub Actions workflow.

Entry / Exit Criteria

Entry Criteria: a build compiles cleanly via make; the reference Ubuntu/GCC test environment is available; test data (sample files, command scripts) is prepared.

Exit Criteria: 100% of planned test cases executed; zero critical defects open (crashes, memory-safety errors); all High-priority functional and security requirements pass.

5.1 Security Validation

- Verify production code in src/ and include/ does not use system(), popen() or equivalent shell-interpolation calls to execute user commands (SEC-01), using code review and a static source check. Test harnesses may use system() to launch the compiled shell for integration testing.
- Boundary/fuzz testing of input length around the 1024-character limit (SEC-02), including deliberately oversized and malformed lines.
- Verify environment-variable values are never printed or logged except via an explicit env call (SEC-03).
- Signal-handling test: Ctrl+C during a foreground external command must not terminate the shell itself (SEC-04, FR-PE-07).
- Redirection permission test: files are created or modified only when the user explicitly supplies <, > or >>, and only with the invoking user's own OS permissions (SEC-05).
- Run the functional test suite under Valgrind and/or an AddressSanitizer-instrumented build and inspect the report for leaks, use-after-free and invalid accesses (SEC-06). This validation remains pending until executed.

6. Test Environment

- Hardware: standard development laptop or VM running the reference test environment.
- Software: Ubuntu/Linux, GCC, GNU Make; custom C test harnesses. Valgrind and/or AddressSanitizer and cppcheck are optional validation tools to be used when available. Check and CMocka are not project dependencies.
- Tools: GitHub Actions (CI), GitHub Projects or Jira (defect tracking and sprint board), Git/GitHub (version control).
- Test Data: sample command scripts, dummy files for redirection tests, PATH-available utilities used as external test commands (ls, cat, wc, sort, grep, uniq).

7. Test Schedule

Milestone	Timing
Test case design	Alongside requirements and design (WP1–WP2)
Per-module unit tests	During each module's implementation sprint (WP3–WP6)
Integration testing	(WP7)
System / acceptance testing	(WP8)

8. Test Deliverables

- This Test Plan (STP)
- Detailed test cases (13.2 of this document) and their custom C test-harness source
- Test scripts and test data
- Test execution logs (CI output)
- Defect reports (GitHub Issues)
- Test Summary Report (final validation)

9. Roles and Responsibilities

Role	Name	Responsibility
F1 owner & tester	Rekha Dhorigol	Design and execute F1 test cases; reviews F4 tests
F2 owner & tester	Rohan Rampura Praveen Kumar	Design and execute F2 test cases; reviews F1 tests
F3 owner & tester	Samika Ojha	Design and execute F3 test cases; reviews F2 tests
F4 owner & tester	Sanjana Subrahmanya Aithal	Design and execute F4 test cases; reviews F3 tests

10. Risks and Mitigation

Risk	Mitigation
Timing-based performance tests (NFR-01/02) are flaky on shared/loaded machines	Run on the same reference machine; report the average of 3 runs (SRS 8)
Pipe/fd leaks causing a hung test run	FR-IO-07 descriptor-closing discipline; resource test TC-IO-02 explicitly checks fd growth
Zombie processes from background-job tests interfering with later tests	FR-PE-06 reaping verified in TC-PE-02 before subsequent tests run
Valgrind/ASan unavailable in a contributor's local environment	Run AddressSanitizer and/or Valgrind on an available reference machine; current GitHub Actions CI runs make clean, make and make test but does not run memory-safety instrumentation.

11. Assumptions & Dependencies

- The GCC/Make toolchain is available in the CI environment.
- PATH-available external utilities (ls, cat, grep, sort, uniq, wc) are present in the test environment.
- Valgrind and/or AddressSanitizer are available on the reference test machine.

12. Suspension & Resumption Criteria

Suspend testing if: the build fails to compile, or the reference test environment is unavailable for more than 4 hours.

Resume testing if: the build is fixed, or the reference environment is restored.

13. Test Case Management & Traceability

The RTM below maps each SRS requirement to the test case(s) that verify it, consistent with the Requirement Traceability Matrix in SRS Appendix C.

13.1 Traceability Matrix (RTM)

Req ID	Requirement (short)	Test Case ID(s)
FR-CI-01, FR-CI-04	Prompt display; core built-ins	TC-CI-01
FR-CI-02, FR-CI-06, SEC-02	Tokenization; 1024-char boundary	TC-CI-02
FR-PE-01, FR-PE-02, FR-PE-03	Foreground external execution	TC-PE-01
FR-PE-04, FR-PE-06	Background execution; child reaping	TC-PE-02
FR-IO-01, FR-IO-02, FR-IO-03	Output/append/input redirection	TC-IO-01
FR-IO-04, FR-IO-05, FR-IO-07	Multi-stage pipeline; descriptor cleanup	TC-IO-02
FR-HE-01, FR-HE-02, FR-HE-03	History recording, display, eviction	TC-HE-01
FR-HE-04, FR-HE-05, FR-HE-06, FR-HE-07	export/unset/env; child visibility	TC-HE-02
SEC-01, SEC-05	No shell interpolation; explicit redirection only	TC-SEC-01
SEC-02, SEC-06	Bounded input; memory safety	TC-SEC-02
NFR-01, NFR-02	Prompt-response and startup latency	TC-NFR-01
NFR-03, NFR-04	Pipeline throughput; built-in discoverability	TC-NFR-02
NFR-05	Per-module maintainability (≥70% coverage)	Per-module coverage report (pending; current CI does not collect coverage) a discrete runtime test case

13.2 Detailed Test Cases

Twelve test cases are defined below, covering all four functional areas, security requirements and measurable non-functional requirements. Actual Result and Pass/Fail remain TBD until the corresponding tests are executed and evidence is recorded.

TC-CI-01 — Prompt display and core built-in execution

Field	Detail
Requirement(s)	FR-CI-01, FR-CI-04
Precondition	Shell is built and launched
Steps	1) Launch the shell 2) Observe the prompt 3) Type 'pwd' and press Enter
Test Data	Command: pwd
Expected Result	Prompt 'simple-shell> ' is shown; pwd prints the current working directory; prompt redisplays
Actual Result	TBD
Pass/Fail	TBD

TC-CI-02 — Tokenization and input-length boundary

Field	Detail
Requirement(s)	FR-CI-02, FR-CI-06, SEC-02
Precondition	Shell is running
Steps	1) Enter a command line of exactly 1024 characters 2) Enter a command line of 1025 characters
Test Data	1024-char echo command; 1025-char echo command
Expected Result	The 1024-char line executes normally; the 1025-char line is rejected with an error and the shell does not crash or corrupt state
Actual Result	TBD
Pass/Fail	TBD

TC-PE-01 — Foreground external command execution

Field	Detail
Requirement(s)	FR-PE-01, FR-PE-02, FR-PE-03
Precondition	Shell is running
Steps	1) Enter 'ls -l' 2) Observe output and prompt timing
Test Data	Command: ls -l
Expected Result	External command output is shown; the shell waits for completion before redisplaying the prompt
Actual Result	TBD
Pass/Fail	TBD

TC-PE-02 — Background execution and zombie-free reaping

Field	Detail
Requirement(s)	FR-PE-04, FR-PE-06
Precondition	Shell is running
Steps	1) Enter 'sleep 2 &' 2) Immediately enter 'pwd' 3) Wait 3 seconds 4) Check for zombie processes via ps
Test Data	Command: sleep 2 &
Expected Result	Prompt returns immediately after '&'; pwd executes without waiting; no zombie 'sleep' process remains after completion
Actual Result	TBD
Pass/Fail	TBD

TC-IO-01 — Output, append and input redirection

Field	Detail
Requirement(s)	FR-IO-01, FR-IO-02, FR-IO-03
Precondition	Shell is running
Steps	1) Run 'echo hi > out.txt' 2) Run 'echo bye >> out.txt' 3) Run 'wc -l < out.txt'
Test Data	File: out.txt (created during the test)
Expected Result	out.txt contains 'hi' then 'bye' on two lines; wc -l reports 2
Actual Result	TBD
Pass/Fail	TBD

TC-IO-02 — Multi-stage pipeline and descriptor cleanup

Field	Detail
Requirement(s)	FR-IO-04, FR-IO-05, FR-IO-07
Precondition	Shell is running
Steps	1) Run 'ls | grep .c | wc -l' (3-stage pipeline) 2) Repeat the same pipeline 20 times in a loop 3) Check open fd count for the shell process via /proc/<pid>/fd
Test Data	Command: ls | grep .c | wc -l
Expected Result	Correct count is printed each time; after 20 repetitions, the shell's open-fd count has not grown (no descriptor leak)
Actual Result	TBD
Pass/Fail	TBD

TC-HE-01 — History recording, display and 50-entry eviction

Field	Detail
Requirement(s)	FR-HE-01, FR-HE-02, FR-HE-03
Precondition	Shell is running
Steps	1) Run 55 distinct simple commands 2) Run 'history'
Test Data	55 sequential simple commands
Expected Result	history displays the most recent 50 entries, numbered in order; the oldest 5 are evicted
Actual Result	TBD
Pass/Fail	TBD

TC-HE-02 — export / unset / env and child visibility

Field	Detail
Requirement(s)	FR-HE-04, FR-HE-05, FR-HE-06, FR-HE-07
Precondition	Shell is running
Steps	1) Run 'export FOO=bar' 2) Run 'env' and check FOO is present 3) Run an external command that reads $FOO 4) Run 'unset FOO' 5) Run 'env' again
Test Data	Variable: FOO=bar
Expected Result	FOO=bar appears in env after export and is visible to the external subprocess; FOO is absent from env after unset
Actual Result	TBD
Pass/Fail	TBD

TC-SEC-01 — No shell interpolation; explicit-only redirection

Field	Detail
Requirement(s)	SEC-01, SEC-05
Precondition	Shell is running
Steps	1) Enter a command containing shell metacharacters as a literal argument, e.g. echo "a; rm -rf /tmp/x" 2) Confirm no secondary command executes 3) Confirm no file is created or modified unless </>/>> is explicitly used
Test Data	Command: echo "a; rm -rf /tmp/x"
Expected Result	The string is printed literally as one argument; no injected command executes; no unintended file operation occurs
Actual Result	TBD
Pass/Fail	TBD

TC-SEC-02 — Memory-safety verification

Field	Detail
Requirement(s)	SEC-02, SEC-06
Precondition	Shell built with ASan/Valgrind instrumentation
Steps	1) Run the full functional test suite (TC-CI-01..TC-HE-02) under Valgrind/ASan 2) Inspect the report
Test Data	Full functional test suite
Expected Result	Zero memory leaks, use-after-free, or invalid-access errors reported
Actual Result	TBD
Pass/Fail	TBD

TC-NFR-01 — Prompt-response and startup latency

Field	Detail
Requirement(s)	NFR-01, NFR-02
Precondition	Reference Ubuntu/GCC test environment
Steps	1) Measure time from launch to first prompt (3 runs, average) 2) Measure time from foreground command completion to next prompt (3 runs, average)
Test Data	Commands: pwd, echo hi
Expected Result	Startup ≤ 1s and prompt redisplay ≤ 1s, both averaged over 3 runs on the reference machine
Actual Result	TBD
Pass/Fail	TBD

TC-NFR-02 — Pipeline throughput and built-in discoverability

Field	Detail
Requirement(s)	NFR-03, NFR-04
Precondition	Reference environment; a first-time test user for the usability check
Steps	1) Run a 5-stage lightweight pipeline 10 times and confirm no hang/deadlock 2) Ask a first-time user to run at least 3 of (cd, pwd, echo, exit, help) unaided within 2 minutes
Test Data	Pipeline: cat file | grep x | sort | uniq | wc -l
Expected Result	No pipeline run hangs or deadlocks; the first-time user completes ≥3 built-ins within 2 minutes without external documentation
Actual Result	TBD
Pass/Fail	TBD

14. Test Metrics & Reporting

Metrics collected: % of test cases executed, % passed/failed, defect count by severity, defect aging, and requirement coverage (% of FR/NFR/SEC IDs with at least one associated test case — currently 100% per 13.1).

Reports: a status summary after each sprint's test activity, and a final Test Summary Report (Final Report, System Validation & Demonstration).

15. Approvals

Role	Name	Signature / Date
Course Coordinator	Dr. Sapna V M	